netsim: Convergence-First Network Simulator

Technical Reference

Simon Knight

March 2026

Abstract. netsim is a tick-based network simulator written in Rust that bridges the gap between algorithmic configuration analysis and container-based network emulation. It provides protocol-accurate simulation of OSPF (v2/v3), IS-IS, BGP (including EVPN/VXLAN and L3VPN), MPLS/LDP, RSVP-TE, BFD, LACP, LLDP, RIPv2/RIPng, STP, VRRP v2/v3, and DHCP Relay with deterministic convergence detection at 486-device scale. This technical report documents the system's architecture, design rationale, data model, and usage. It is intended for engineers evaluating netsim for network validation, contributors to the codebase, and researchers interested in the convergence-first simulation model.

Version	Date	Changes
1.0	March 2026	Initial release
1.1	March 2026	Add RIP, STP, VRRP, DHCP Relay, path tracing, traceroute, dig, iperf, CoPP, E2E hardening

Contents

1 Introduction

netsim is a network simulator that models routing protocol behaviour at packet granularity with deterministic, reproducible results. It accepts a single YAML file describing a network topology—devices, links, protocol configurations, traffic generators, failure patterns, and assertions—and simulates the network tick by tick until all routing protocols converge. It implements 15 protocol families spanning link-state routing (OSPF v2/v3, IS-IS), distance-vector routing (RIPv2/RIPng), path-vector routing (BGP-4 with EVPN), label switching (MPLS/LDP, RSVP-TE), L2 infrastructure (STP, LACP, LLDP, VXLAN), first-hop redundancy (VRRP v2/v3), address management (DHCP Relay), and fast failure detection (BFD). It replaces the need for container-based emulation [4] in scenarios where determinism and speed matter more than vendor-specific NOS fidelity.

Two core design principles explain every subsequent decision in this document:

1. **Convergence is a first-class time coordinate.** Routing convergence is not a side effect of "running long enough." It is a precisely defined predicate (the simultaneous stability of 11 protocol states) that is observable, addressable, and scriptable. Events, assertions, and chaos engineering all reference convergence as a time anchor (`at: converged + 500ms`).
2. **Determinism over fidelity.** netsim models RFC behaviour, not vendor-specific implementation quirks. All timers are integer ticks, all RNGs are seeded, and all protocol synchronisation uses snapshot-then-apply rather than per-message queuing. This trades the last mile of realism for bit-identical reproducibility across runs.

This document is intended for three audiences:

- **Network engineers** evaluating netsim for pre-deployment validation in CI/CD pipelines.
- **Contributors** to the netsim codebase who need to understand architectural decisions before modifying the system.
- **Researchers** interested in the convergence-first simulation model and its implications for network testing.

Familiarity with IP routing protocols (OSPF, BGP) and basic Rust syntax is assumed. Experience with network emulation tools (Containerlab, ns-3) is helpful but not required.

Section 2 defines the core data model (devices, wires, interfaces). Section 3 presents the system architecture and the tick pipeline. Section 4 explains the convergence predicate in detail. Section 5 covers the forwarding architecture (RIB/FIB, ECMP). Section 6 surveys all 15 protocol families (including RIP, STP, VRRP, and DHCP Relay). Section 7 describes the YAML topology format and chaos engineering. Section 8 covers EVPN/VXLAN at simulation abstraction. Section 9 documents the CLI tool, interactive daemon, diagnostic commands (traceroute, dig, iperf), and path tracing engine. Section 10 provides end-to-end usage examples. Section 11 catalogues the 40+ example topologies shipped with netsim and their protocol coverage. Section 12 documents performance characteristics, benchmarking infrastructure, and scalability. Section 13 lists known limitations honestly. Section 14 outlines future work.

Readers wanting to get started quickly should read sections 2 and 10, then refer to individual sections as needed.

2 Data Model

This section defines the core abstractions that all other components build upon.

2.1 Devices

netsim models five device types, all implementing the `Device` trait:

Table 1: Device types and their roles.

Type	Rust type	Description
Router	`Router`	Full L3 device with RIB/FIB, protocol daemons (OSPF v2/v3, BGP, IS-IS, RIP, MPLS, RSVP-TE, BFD, LLDP, LACP, VRRP, DHCP Relay), bridge domains (STP), VRFs
Host	`Host`	L3 endpoint with a default gateway; generates and sinks traffic
Switch	`Switch`	L2 device with MAC learning and VLAN support
Hub	`Hub`	L1 device that floods all frames to all ports
Wire	`Wire`	Point-to-point link with latency, loss, jitter, and capacity modelling

Note

A sixth device type, `DnsServer`, implements the `Device` trait for internal DNS resolution (used by the `dig` command). It is not stored in the engine's type-segregated vectors and cannot be instantiated from YAML—it exists solely to provide DNS query/response handling within the simulation.

Design Decision

Why five types instead of a generic "Node"?
Type-segregated storage (`Vec<Option<Box<Router>>>`, `Vec<Option<Box<Host>>>`, etc.) enables Rayon's `par_iter_mut` to process each device type independently without `unsafe` code [12]. A single `Vec<Box<dyn Device>>` would require `unsafe` downcasting or runtime locks for parallel mutation. The five-type design trades some code duplication for zero-cost safe parallelism.

Each device is identified by a `DeviceId(u32)` and contains a vector of interfaces. Interfaces are identified by `InterfaceId(u32)` and carry IPv4/IPv6 addresses, MAC addresses, and link-state information.

2.2 Wires

A `Wire` connects exactly two interfaces on different devices. It models:

- **Latency**: Frames spend a configurable number of ticks in transit (implemented as a `VecDeque` of in-flight frames).
- **Packet loss**: Probabilistic drop using a seeded PRNG.
- **Jitter**: Random variation added to the base latency.
- **Capacity**: Interface bandwidth for utilisation tracking (added in Phase 118).

2.3 Identifiers

Table 2: Identifier types used throughout netsim.

Type	Inner	Usage
`DeviceId`	`u32`	Index into device storage vectors
`InterfaceId`	`u32`	Index into a device's interface vector
`WireId`	`u32`	Index into the wire storage vector
`RouterId`	`Ipv4Addr`	OSPF/BGP router identifier

Note

`InterfaceId(u32::MAX)` is a sentinel value used for null routes (traffic explicitly dropped). This appears in the FIB when a route has no valid next-hop.

3 System Architecture

netsim processes a simulation in three stages: *build*, *run*, and *output*.

3.1 Build Stage

The Builder reads a YAML topology file and constructs the in-memory simulation state:

1. **Schema validation**: The YAML is validated against the topology schema. Missing required fields, unknown device types, and invalid IP addresses are caught here.
2. **Device construction**: Each device entry creates a `Router`, `Host`, `Switch`, or `Hub` with configured interfaces.
3. **Wire creation**: Each link entry creates a `Wire` connecting two interfaces. Latency, loss, and jitter are applied from named impairment profiles or inline values.
4. **Pattern resolution**: Pattern-based selectors (`by_tier: spine`, `name_pattern: "r*"`) are expanded to concrete device/link lists.
5. **Event scheduling**: Failure patterns (`FailurePattern`) are expanded into a sequence of `ScheduledEvents`. Convergence-relative events (`at: converged + 500ms`) are deferred until convergence is detected at runtime.

Design Decision

Why separate build from run?
Separating build from run allows the Builder to perform expensive validation (schema checks, pattern expansion, event scheduling) once, producing an immutable simulation configuration. The engine then runs without re-validating on every tick. This also enables future features like topology caching and incremental rebuilds.

3.2 Run Stage: The Tick Loop

The engine advances simulation time one tick at a time (default: 1 ms/tick). It uses a **Zero-Copy Structure-of-Arrays (SoA)** architecture to maximise parallel execution performance and CPU cache hit rates. Device storage uses stable memory locations (SlotMaps), allowing pointers and indices to persist across ticks without the $O(N)$ reconstruction overhead typical of object-oriented designs.

Each tick executes eight phases in strict order:

Table 3: The 8-phase tick pipeline.

Phase	Name	Description
P0	Drain commands	Dequeue pending gRPC commands for deterministic delivery
P1	Device→Wire	Move outbound frames from device output queues to connected wires
P2	Tick devices	Run all protocol state machines (OSPF SPF, BGP decision, BFD timers, etc.). Parallel via Rayon if ≥500 devices
P2.5	EVPN sync	Snapshot-then-apply EVPN Loc-RIB synchronisation
P3	Tick wires	Advance in-transit frames by one tick; apply impairments (loss, jitter)
P4	Wire→Device	Deliver arrived frames to destination device input queues
P5	Convergence check	Evaluate the composite convergence predicate (section 4)
P6	Hooks & quiescence	Check quiescence; fire convergence hooks; schedule deferred events
P7	Export	Write PCAP, BMP, NetFlow snapshots

Note

The phase ordering is load-bearing. P1 must precede P2 so that devices process frames that were sent in the *previous* tick, not the current one. P2.5 must follow P2 so that EVPN updates reflect the current tick's BGP processing. Reordering phases will break protocol correctness.

3.3 Determinism Guarantees

Three properties ensure that two runs of the same topology produce identical results:

1. **Integer ticks**: All protocol timers are expressed in integer ticks. No floating-point arithmetic appears in timing-sensitive code paths.
2. **No shared mutable state during P2**: During device ticking, each device reads only from its own input queue and writes only to its own output queue. Rayon's `par_iter_mut` is safe because no two devices access the same memory.
3. **Seeded PRNGs**: All random number generators (ECMP hashing, probabilistic failure injection, jitter) use deterministic seeds derived from the topology configuration.

3.4 Adaptive Parallelism

Design Decision

Problem: Rayon's thread-pool scheduling has non-trivial overhead for small workloads.
Options considered: (1) Always parallel. (2) Always serial. (3) Adaptive threshold.
Decision: Adaptive threshold at 500 devices. Below 500, serial iteration. At or above 500, Rayon `par_iter_mut`.
Trade-off: The threshold is a compile-time constant, not auto-tuned. A topology with 499 devices runs serially even if the machine has 64 cores. Benchmarking on an 8-core CPU shows that serial execution is 2–3% faster for 486 devices, validating the 500-device threshold as a reasonable heuristic for modern hardware.

3.5 Output Stage

After the simulation terminates (convergence detected, max ticks reached, or user abort), netsim exports:

- **Routing matrix** (JSON): Per-device FIB entries with next-hops, metrics, and protocol origins.
- **PCAP captures**: Per-interface packet captures for offline analysis in Wireshark.
- **BMP feeds**: BGP Monitoring Protocol messages for integration with route collectors.
- **NetFlow/IPFIX**: Flow-level traffic summaries.
- **Convergence report**: Tick-by-tick protocol state transitions showing exactly when each protocol stabilised.
- **Assertion results**: Pass/fail for all topology-defined assertions.
- **Topology diff**: Before/after comparison of routing state across topology changes.
- **Capacity report**: Per-interface utilisation against configured bandwidth.
- **Path trace**: Hop-by-hop forwarding path analysis with anomaly detection (loops, blackholes, asymmetry).
- **FIB export**: Routing state in netauto/fib/v1.0 schema for integration with external network analysis tools.

4 Convergence Detection

Convergence detection is netsim's most distinctive feature. This section explains the design in full.

4.1 Problem

Network simulators typically determine "convergence" by one of two methods:

1. **Timeout**: Run for a fixed number of seconds/ticks and assume convergence. Fragile—too short misses slow convergence; too long wastes time.
2. **Quiescence**: Stop when no more packets are in flight. Incorrect—LLDP, BFD, and other periodic protocols never stop sending packets.

Neither method answers the operator's actual question: "Have all routing protocols finished computing and installed stable forwarding state?" The problem is well-documented: delayed Internet routing convergence [8] can cause transient loops and black holes that persist for seconds after a topology change.

4.2 Design

netsim defines convergence as the simultaneous stability of 11 independent protocol states, sustained for N consecutive ticks (default $N = 3$):

$$converged(t) = \bigwedge_{i=1}^{11} (\forall\, \tau \in [t{-}N{+}1,\ t] \ :\ stable_i(\tau)) \tag{1}$$

The 11 stability flags are:

#	Flag	Stable when…
1	FIB	No FIB entry changed this tick
2	VRF FIBs	No per-VRF FIB entry changed this tick
3	MPLS LFIB	No label forwarding entry changed this tick

#	Flag	Stable when…
4	LDP bindings	No LDP label binding changed this tick
5	VPNv4 Loc-RIB	No VPNv4 route changed this tick
6	BGP Loc-RIB	No BGP route changed this tick
7	EVPN Loc-RIB	No EVPN route changed this tick
8	OSPF neighbours	No OSPF adjacency changed state this tick
9	BGP sessions	No BGP session changed state this tick
10	RSVP tunnels	No RSVP-TE tunnel changed state this tick
11	IS-IS adjacencies	No IS-IS adjacency changed state this tick

Design Decision

Why exclude observability protocols?
PCAP, BMP, and NetFlow generate continuous background traffic that never stops. Including them in the convergence predicate would prevent convergence from ever being detected. Similarly, LLDP sends periodic hellos every 30 seconds regardless of routing state—it is explicitly excluded from quiescence checks via `EtherType::is_control_plane() == false`.

Note

Protocols added after the initial 11 flags—RIP, STP, VRRP, and DHCP Relay—do not require additional convergence flags. RIP installs routes into the RIB, which updates the FIB (tracked by flag #1). STP modifies port forwarding state on Switches and Router bridge domains (which affects frame delivery but not the routing FIB). VRRP changes the active gateway, which manifests as a FIB change on the relevant router. DHCP Relay forwards packets without modifying routing state. The existing 11-flag predicate therefore covers these protocols transitively.

4.3 Convergence as a Time Coordinate

Once convergence is detected at tick t_c, netsim makes t_c available as a named time anchor. Subsequent events can reference it:

Listing 1: Convergence-relative event scheduling.

```
events:
  - at: converged + 100    # 100 ticks after convergence
    action: link_down
    link: r1-r2

  - at: converged + 500ms  # 500ms after convergence
    device: PE1
    command: show bgp summary

assertions:
  - type: convergence_time
    max_ticks: 200         # fail if convergence takes > 200 ticks
```

This enables three workflows that are impossible with timeout-based convergence:

1. **CI-gatable assertions**: “Fail this pipeline if convergence takes longer than 200 ticks after a link failure.”
2. **Convergence-relative chaos**: “Inject a second failure 100 ticks after convergence from the first failure.”
3. **Post-convergence inspection**: “Run `show bgp summary` on PE1 exactly 500 ms after convergence to verify the BGP table.”

5 Forwarding Architecture

5.1 RIB/FIB Separation

Each router maintains two distinct data structures:

- The **RIB** (Routing Information Base) stores all routes learned from all protocols, tagged with their source protocol, administrative distance, and metric.
- The **FIB** (Forwarding Information Base) stores only the best routes selected from the RIB. The FIB is what the forwarding engine consults when processing a packet.

> **Design Decision**
>
> **Why separate RIB and FIB?**
> Real routers maintain this separation because the control plane (protocol daemons) and data plane (forwarding ASIC) operate independently. netsim mirrors this to ensure that protocol interactions (e.g., OSPF and BGP both advertising the same prefix) are resolved correctly via administrative distance, and to enable accurate modelling of RIB→FIB installation delays.

5.2 Dual-Index FIB

The FIB uses two co-maintained data structures:

1. A **binary trie** for longest-prefix-match (LPM) lookups. Implemented as `Vec<TrieNode>` where each `TrieNode` has `[Option<usize>; 2]` children and an optional `FibEntry`. LPM walks the trie bit by bit from MSB to LSB, tracking the last seen entry. Worst case: 32 steps for IPv4.
2. A **HashMap** (`HashMap<Ipv4Net, FibEntry>`) for $O(1)$ exact-match lookups during FIB update change detection.

To efficiently handle high-churn routing environments (e.g., Internet-scale BGP), netsim implements **Incremental FIB Updates**. Adding or removing a prefix from the RIB results in an $O(\log N)$ update to the Trie rather than an $O(N)$ full FIB rebuild. Atomic multi-prefix updates allow hundreds of route changes to be committed to the FIB in a single batch, drastically improving simulator convergence times on large topologies.

> **Design Decision**
>
> **Problem**: FIB updates must not trigger spurious convergence resets. If a protocol daemon re-announces an unchanged route, and the FIB's change flag is set, convergence detection restarts unnecessarily.
> **Options considered**: (1) Trie-only FIB with entry comparison on every update ($O(W)$ per lookup + $O(W)$ per update comparison). (2) HashMap-only FIB ($O(1)$ updates but $O(n)$ LPM via linear scan). (3) Dual-index with trie for LPM and HashMap for change detection.
> **Decision**: Dual-index. The HashMap enables $O(1)$ equality checking during updates (`old_entry == new_entry?`), and the trie provides $O(W)$ LPM for forwarding. Both are updated atomically via `replace_entries`.
> **Trade-off**: Double memory for the FIB. In practice, FIB memory is negligible compared to protocol state (BGP RIB, OSPF LSDB).

5.3 ECMP

When the RIB installs multiple equal-cost routes to the same prefix, the FIB stores all next-hops. At forwarding time, a flow-consistent hash selects among them:

Listing 2: ECMP hash computation.

```
fn ecmp_hash(src_ip: u32, dst_ip: u32, nhops: usize) -> usize {
    (src_ip.wrapping_add(dst_ip.rotate_left(16))) as usize % nhops
```

```
}
```

This ensures all packets of a given (src, dst) flow traverse the same path, while distributing distinct flows across available next-hops.

> **Warning**
>
> The ECMP hash does not include L4 ports. This means all traffic between a given (src_ip, dst_ip) pair follows the same path regardless of application. This is simpler than real router implementations (which typically hash on the 5-tuple) but sufficient for routing-level validation.

6 Protocol Implementations

netsim implements the following routing and infrastructure protocols. Each implementation targets RFC-correct behaviour at tick granularity, not vendor-specific quirks.

Table 5: Protocol coverage.

Protocol	RFCs	Scope
OSPF v2	2328 [13]	Areas, stub/NSSA, virtual links, SPF with incremental recalculation
IS-IS	ISO 10589	L1/L2, wide metrics, TLV extensibility
BGP-4	4271 [15]	iBGP, eBGP, route reflection, confederations, communities, AS-path filtering
BGP EVPN	7432 [16]	Type-2/3/5, RT import/export, ARP suppression
MPLS/LDP	5036 [1]	Label distribution, PHP, ECMP label stacking
RSVP-TE	3209	Explicit paths, bandwidth reservation, FRR
BFD	5880 [7]	Async mode, microsecond-granularity timers (mapped to ticks)
LACP	802.3ad	Fast/Slow timers (100/3000ms), LAG bundling, actor/partner state machines
LLDP	802.1AB	Periodic hellos, neighbour table, TLV parsing
VXLAN	7348 [9]	Encap/decap, head-end replication
RIPv2	2453 [10]	Periodic/triggered updates, split horizon, 15-hop metric limit, route poisoning
RIPng	2080 [11]	IPv6 distance-vector routing
STP	802.1D	Root bridge election, port roles (Root/Designated/Blocking), topology change detection
VRRP v2/v3	3768 [6], 5798 [14]	Master/Backup election, virtual IP, tracked interfaces, priority decrements, virtual MAC (00:00:5E:00:01:VRID)
DHCP Relay	2131 [5]	Broadcast-to-unicast conversion, multiple helper addresses, GIADDR population, Option 82 (Circuit ID, Remote ID)
OSPFv3	5340 [3]	IPv6 OSPF with link-local addressing

> **Note**
>
> All protocol timers (OSPF dead interval, BGP hold time, BFD detection multiplier, LACP fast/slow, RIP update interval, VRRP advertisement interval) are converted to integer tick counts at build time. A 10-second OSPF dead interval at 1 ms/tick resolution becomes 10,000 ticks.

6.1 OSPF

The OSPF implementation follows RFC 2328. Each router runs an independent OSPF instance with:

- Neighbour state machine (Down → Init → 2-Way → ExStart → Exchange → Loading → Full)
- Link-State Database (LSDB) synchronised via Database Description and LSA exchange
- SPF computation triggered on LSDB change (not on every tick)
- Area support: backbone (Area 0), stub, NSSA, virtual links

6.2 BGP

The BGP implementation supports both iBGP and eBGP with:

- Full FSM (Idle → Connect → OpenSent → OpenConfirm → Established)
- Route reflection (cluster-id, originator-id, no client-to-client)
- Best-path selection (13-step decision process matching Cisco IOS)
- Community, extended community, and large community support
- AS-path prepending, local-pref, MED
- Address families: IPv4 unicast, VPNv4, EVPN

6.3 MPLS and RSVP-TE

Label switching is implemented with:

- MPLS label stack with push/swap/pop operations
- LDP for automatic label distribution following the IGP topology
- RSVP-TE for explicit label-switched paths with bandwidth constraints
- Penultimate hop popping (PHP)
- L3VPN with VRF import/export using VPNv4 route targets

6.4 BFD, LACP, LLDP

Infrastructure protocols:

- **BFD**: Bidirectional Forwarding Detection for fast failure detection. BFD session state changes trigger IGP/BGP convergence immediately rather than waiting for hold-timer expiry.
- **LACP**: Link Aggregation Control Protocol with tick-aligned timers (Fast: 100/300 ticks, Slow: 3000/9000 ticks). LAG bundles present as a single logical interface to the routing protocols.
- **LLDP**: Periodic neighbour discovery. Processed by `tick_lldp()` in `tick_control()`. Excluded from quiescence checks because it never stops sending.

6.5 RIP

netsim implements both RIPv2 (RFC 2453 [10], IPv4) and RIPng (RFC 2080 [11], IPv6). Key features:

- **Hop-count metric**: 15-hop limit with 16 as infinity (unreachable). Split horizon with poisoned reverse prevents count-to-infinity.

- **Periodic updates**: Default 30-second interval with triggered updates on topology changes for faster convergence.
- **Interface activation**: Enabled per-interface via `rip: true` in the YAML topology.

> **Design Decision**
>
> **Why include RIP in a modern simulator?**
> RIP is rarely deployed in new networks, but it remains valuable for three reasons: (1) it is the simplest routing protocol to understand, making netsim useful for educational settings; (2) legacy networks still run RIP in stub areas; (3) RIP's distance-vector behaviour (count-to-infinity, split horizon) exercises different convergence dynamics than OSPF's link-state SPF, broadening the convergence predicate's test coverage.

6.6 STP

The Spanning Tree Protocol implementation provides L2 loop prevention for Switch devices:

- **Root bridge election**: Lowest bridge ID wins. BPDUs propagate root bridge identity across the switched domain.
- **Port roles**: Root, Designated, and Blocking roles are computed from received BPDUs.
- **State transitions**: Disabled → Listening → Learning → Forwarding, with configurable forward delay.
- **Topology change detection**: TCN BPDUs trigger MAC table flushes to prevent stale forwarding.

6.7 VRRP

Virtual Router Redundancy Protocol (RFC 3768 [6] for v2, RFC 5798 [14] for v3) provides first-hop gateway redundancy:

- **Master/Backup election**: Priority-based (default 100, range 1–254); highest priority becomes Master.
- **Virtual IP**: A shared IP address floats between Master and Backup routers. Hosts configure this as their default gateway.
- **Tracked interfaces**: Interface state changes adjust VRRP priority dynamically (configurable decrements), enabling upstream-aware failover.
- **Preemption**: Configurable—a higher-priority router can reclaim Master status when it recovers.
- **Virtual MAC**: Standard 00:00:5E:00:01:*VRID* address, avoiding gratuitous ARP storms on failover.
- **IPv4 and IPv6**: Both address families supported via VRRPv3.

6.8 DHCP Relay

The DHCP Relay Agent (RFC 2131 [5]) enables hosts in remote subnets to obtain addresses from a centralised DHCP server:

- **Broadcast-to-unicast**: Converts DHCP Discover/Request broadcasts into unicast packets to configured helper addresses.
- **Multiple servers**: Each interface can list multiple `helper_addresses`, forwarding to all.
- **GIADDR**: The relay populates the Gateway IP Address field so the server selects the correct subnet pool.
- **Option 82**: Appends Relay Agent Information (Circuit ID and Remote ID sub-options) for downstream accounting. Pre-existing Option 82 from untrusted sources is dropped for security.

- **Statistics**: Per-interface counters for requests forwarded, replies forwarded, and drops.

Design Decision

Why DHCP Relay without a full DHCP server?
netsim models the *relay* path, not the server's address pool logic. This is sufficient to validate enterprise campus designs where DHCP Relay traversal through firewalls and VRFs is the common failure mode. A full DHCP server would add complexity without improving the relay-path validation that operators care about.

7 Declarative Topology & Chaos Engineering

7.1 YAML Topology Format

A netsim topology is a single YAML file with the following top-level sections:

Listing 3: Top-level YAML structure.

```yaml
# Metadata
name: "dc-fabric-486"
tick_ms: 1  # optional, default 1ms

# Network elements
devices: [...]
links: [...]

# Protocol configuration
ospf: {...}    # optional
bgp: {...}     # optional
# ... other protocols

# Operational
traffic: [...]             # optional
impairment_profiles: [...]  # optional
failure_patterns: [...]     # optional
events: [...]              # optional
assertions: [...]          # optional
```

7.2 Pattern-Based Selectors

Rather than configuring each device individually, netsim supports selectors that match groups of devices or links:

Listing 4: Pattern-based selectors.

```yaml
impairment_profiles:
  - name: wan-latency
    selector:
      by_tier: [core, edge]   # match by device tier
    latency_ms: 10
    loss_percent: 0.01

  - name: access-links
    selector:
      name_pattern: "access-*"  # match by name glob
    latency_ms: 2

  - name: geographic
    selector:
      by_distance: true    # compute from coordinates
    latency_model: speed_of_light
```

Design Decision

Why selectors instead of per-device configuration?
A 486-device topology would require 630+ link configurations without selectors. Pattern-based selectors reduce this to a handful of profiles, making topologies readable and maintainable. The trade-off is that selectors can match unintended devices if patterns are too broad—netsim logs which devices each selector matched during the build stage to aid debugging.

7.3 Declarative Assertions

To enable automated CI/CD validation, netsim supports declarative assertions that are evaluated after convergence. This allows operators to encode their network intent directly in the topology file:

Listing 5: Declarative assertions.

```
assertions:
  # End-to-end reachability intent
  - type: reachability
    source: host-db
    destination: 10.0.1.1
    expected: true

  # Protocol state invariant
  - type: ospf_neighbor_state
    router: core-spine1
    interface: eth0
    expected_state: Full
```

If any assertion fails, netsim exits with a non-zero status code and provides a detailed diagnostic report, making it ideal for integration into automated infrastructure pipelines.

7.4 Failure Patterns

netsim supports four failure pattern types for chaos engineering [2]:

Table 6: Failure pattern types.

Type	Behaviour
`probabilistic`	At each `check_interval`, fail with probability p. Recovery after n ticks.
`periodic`	Deterministic on/off cycling with configurable up/down durations.
`flapping`	Rapid state oscillation (up→down→up) to stress convergence.
`cascade`	Dependency-triggered chain failures: when a primary link fails, dependent links/devices also fail.

Each `FailurePattern` is expanded into a sequence of `ScheduledEvents` at build time. This cleanly separates *what should happen* (the pattern) from *when it happens* (the schedule), and ensures deterministic replay.

8 EVPN/VXLAN at Simulation Abstraction

Data centre fabric validation is a primary use case for netsim. This section documents the EVPN/VXLAN implementation in detail.

8.1 Problem

Validating a leaf-spine EVPN/VXLAN fabric typically requires Containerlab with FRR or EOS containers. This consumes gigabytes of RAM (each FRR container runs a full Linux userspace), takes minutes to boot, and produces non-deterministic convergence behaviour. An operator wanting to validate "does my EVPN fabric converge correctly after a spine failure?" should not need to wait 3 minutes and hope for consistent results.

8.2 BGP EVPN Control Plane

netsim implements three EVPN NLRI types per RFC 7432 [16]:

- **Type-2** (MAC/IP Advertisement): Advertises host MAC and IP bindings. Used for remote MAC learning and ARP suppression.
- **Type-3** (Inclusive Multicast Ethernet Tag): Establishes the VTEP flood list for BUM (Broadcast, Unknown unicast, Multicast) traffic replication.
- **Type-5** (IP Prefix Route): Advertises IP prefixes for inter-subnet routing across the VXLAN fabric.

Each router maintains per-bridge-domain forwarding databases (FDB) using `BTreeMap<MacAddress, FdbEntry>`. FDB entries are aged out after a configurable timeout.

8.3 ARP Suppression

When a host sends an ARP request, the local VTEP checks its EVPN Type-2 routes for a matching IP→MAC binding. If found, the VTEP replies directly without flooding the ARP request across the VXLAN fabric. This reduces BUM traffic significantly in large fabrics.

8.4 Snapshot-Then-Apply Synchronisation

> **Design Decision**
>
> **Problem**: How should EVPN Loc-RIBs be synchronised across routers in a tick-based simulation?
> **Options considered**:
> 1. **Per-message queuing**: Model individual BGP UPDATE messages with per-session send/receive queues. High fidelity but complex (O(sessions × routes) queue operations per tick) and introduces ordering-dependent non-determinism.
> 2. **Snapshot-then-apply**: At each tick, snapshot all EVPN Loc-RIBs, then have each router apply remote updates from the snapshot.
>
> **Decision**: Snapshot-then-apply (Phase P2.5 in the tick pipeline). This preserves `peer_router_id` metadata to prevent route-reflector-induced flapping while avoiding per-message queue complexity.
> **Trade-off**: Cannot model BGP UPDATE message ordering bugs or per-session backpressure. This is an acceptable trade-off for a tool that targets configuration validation, not BGP implementation testing.

8.5 EVPN Multi-Homing

netsim supports EVPN multi-homing for redundant host connectivity:

- **Ethernet Segment Identifier (ESI)**: Groups interfaces on different VTEPs that connect to the same multi-homed host or switch.
- **Designated Forwarder (DF) election**: Uses service-carving to distribute DF responsibility across VTEPs per VLAN/VNI, preventing duplicate BUM delivery.
- **Split-horizon filtering**: BUM frames received via the VXLAN fabric are not re-flooded back to the ESI from which they originated, preventing loops.

9 CLI Tool & Interactive Daemon

The `netsim` CLI provides both batch execution (run a topology, collect results) and interactive operation (attach to a live simulation, inspect state, inject failures). This section documents both modes.

9.1 CLI Subcommands

The CLI is the primary interface to netsim. It provides five categories of subcommands:

Table 7: CLI subcommand categories.

Category	Commands
Simulation	`run <topology>` — execute a simulation to completion. Options: `--output`, `--format [ascii|json]`, `--max-ticks`, `--convergence-threshold`, `--pcap`, `--scenario`, `--verbose`. `validate <topology>` — check YAML syntax without executing. `example <name>` — print a built-in example topology (`simple`, `ospf-triangle`, `data-center`, `wan-mesh`, `enterprise-campus`, `service-provider`, etc.).
Daemon	`daemon start <topology>` — start a background daemon. Options: `--name`, `--tick-interval [50ms|100ms|1s]`, `--log-level`. `daemon stop <name>` — stop a daemon (`--rm-log` to clean up). `daemon status <name>` — query daemon metrics. `daemon list [--clean]` — list running daemons. `daemon log <name> <preset>` — change log verbosity at runtime.
Interactive	`netsim` (no args) — launch TUI selector (daemon → device → REPL). `attach <daemon> <node>` — open interactive REPL on a device. `exec <daemon> <node> "<cmd>"` — execute a single command.
Analysis	`routing-matrix <daemon>` — extract full routing matrix as JSON. `diff <before.json> <after.json>` — compare two routing matrices. `capacity <topology> <matrix>` — analyse link utilisation. `trace <daemon> <src> <dst>` — trace forwarding path hop-by-hop. `fib <daemon>` — export FIB in netauto/fib/v1.0 schema.

Example: Batch simulation in a CI pipeline

```
# Run topology, output JSON, fail on assertion failures
netsim run topology.yaml --format json --output results.json

# Extract routing matrix from a running daemon
netsim routing-matrix my-fabric --output before.json
```

```
# Compare routing state before and after a change
netsim diff before.json after.json --format ascii
```

9.2 Daemon Architecture

The daemon runs a simulation as a background process, enabling multiple users to attach, inspect, and inject commands concurrently.

Design Decision

Problem: The simulation engine is single-threaded and `!Send` (it holds non-thread-safe state). But gRPC requires an async runtime. How to bridge the two?
Options considered:

1. **Async engine**: Rewrite the engine to be async-compatible. Invasive; every protocol state machine would need refactoring.
2. **Separate OS thread**: Run the engine on a dedicated OS thread; communicate via channels.

Decision: Separate OS thread. The engine runs its tick loop on a dedicated thread. A Tokio async runtime on the main thread hosts the gRPC server. Communication uses `mpsc` channels: commands flow from gRPC handlers to the engine thread, responses flow back.
Trade-off: Requires channel-based indirection for every command. In practice, the overhead is negligible compared to tick processing time.

Process lifecycle. The `daemon start` command forks a background process using the `daemonize` crate. The daemon writes three files to a `.netsim/` directory:

```
.netsim/
  <name>.sock    # Unix Domain Socket for gRPC
  <name>.pid     # Process ID file
  <name>.log     # Structured log with timestamps
```

Using Unix Domain Sockets (UDS) instead of TCP ports avoids port conflicts when running multiple daemons on the same machine and provides implicit access control via filesystem permissions.

Dynamic log control. Log verbosity can be changed at runtime via `daemon log <name> <preset>` without restarting the simulation. Three presets are available:

Table 8: Log presets.

Preset	Filter
`normal`	`info` — standard operational output
`protocol_debug`	OSPF, BGP, IS-IS, BFD at `debug` level; other modules at `info`
`full_trace`	`trace` — all modules, maximum verbosity

This is implemented via the `SetLogPreset` gRPC RPC, which reloads the `tracing` subscriber filter on-the-fly.

9.3 gRPC Service Interface

The daemon exposes five RPCs via Protocol Buffers:

Table 9: gRPC service definition (`proto/netsim.proto`).

RPC	Type	Description
`Exec`	Server stream	Execute a command, stream output lines back
`Attach`	Bidi stream	Interactive REPL session with command/response streaming
`Status`	Unary	Return PID, uptime, tick count, convergence status, average tick duration, device list
`Stop`	Unary	Graceful shutdown
`SetLogPreset`	Unary	Change log verbosity at runtime

The `Status` RPC returns live metrics from shared atomics (tick counter, convergence flag, average tick duration), avoiding lock contention with the engine thread. The response includes a `DeviceInfo` array listing every device by name, type, and interfaces—enabling the TUI to build its device selection menu without parsing the topology file.

9.4 IOS-Style Command Tree

The interactive REPL implements a `BTreeMap`-based command vocabulary tree that provides tab completion and prefix matching.

- **Prefix matching**: Partial commands are expanded to their unique completion (`sh ip ro` → `show ip route`).
- **Ambiguity detection**: When a prefix matches multiple commands (`sh i` → `show interface` or `show ip` or `show isis`), netsim reports an error with all matching completions.
- **Dynamic completion**: Interface names (`eth0`, `eth1`, …) are completed dynamically based on the attached device's actual interfaces.
- **Hints**: The completer shows the first available completion as a grey hint while typing, implemented via the `rustyline` `Hinter` trait.

> **Design Decision**
>
> **Why IOS-style CLI in a simulator?**
> Network engineers have decades of muscle memory for IOS commands. Making the simulator's interactive interface match that mental model reduces the learning curve to zero for the inspection workflow. An engineer who can type `show ip route` on a production router can type the same command in netsim and get comparable output.

9.5 Available Commands

The REPL supports over 50 commands across four categories:

Table 10: Command categories in the interactive REPL.

Category	Commands
Show (30+ variants)	`show ip route [vrf NAME]`, `show ip dhcp relay`, `show ipv6 [route|neighbors]`, `show interfaces`, `show arp`, `show ospf [neighbors|database]`, `show ospfv3 [neighbors|database|interface|route]`, `show bgp [summary|neighbors|vpn]`, `show bgp ipv6 [unicast|summary]`, `show bgp evpn`, `show evpn ethernet-segment`, `show isis [neighbors|database|spf|route|interface]`, `show rip [database|neighbors]`, `show stp [bridge|port]`, `show vrrp [brief|detail]`, `show vxlan [vtep|vni]`, `show mpls [forwarding|lfib]`, `show ldp [bindings|neighbors]`, `show rsvp tunnels`, `show bfd [sessions]`, `show lacp [details]`, `show lldp neighbors`, `show vrf [all|detail|NAME]`, `show sr`, `show gre [tunnels]`, `show bmp`, `show traffic`, `show routing-matrix`, `show trace <destination>`
Diagnostic	`ping <target> [-c COUNT]`, `traceroute <target> [-m MAX_TTL]`, `dig <hostname> [@server]`, `iperf <target> [-d DURATION] [-i INTERVAL]`
Configuration	`route add <prefix> <nexthop>`, `interface [shutdown|no shutdown]`, `mpls [lfib add|ftn add]`, `bgp vpn-originate`, `gre tunnel add`
Session	`help`, `exit`, `quit`

9.6 Ping and Traceroute

The `ping` and `traceroute` commands are notable because they drive *re-entrant simulation stepping*. When a user types `ping 10.0.1.1`, netsim does not simply check reachability in the current routing table—it injects ICMP packets into the simulation and advances the tick loop until replies arrive or a timeout is reached.

- **ping**: Sends ICMP Echo Requests from the attached host, advances the simulation tick by tick, collects Echo Replies, and reports sent/received/lost counts with min/max/avg RTT. Supports `-c|--count` for repeat count (default 4). Only available on Host devices.
- **traceroute**: Sends probes with incrementing TTL, advances the simulation, collects ICMP Time Exceeded and Echo Reply messages, and displays hop-by-hop paths with latency. Supports `-m|--max-ttl` (default 30). Ticks up to 50,000 ticks per probe.

> **Note**
>
> Both commands support device hostname resolution in addition to IP addresses. `ping r1` resolves the hostname `r1` to its first interface IP via an internal device registry, providing a convenient shorthand.

9.7 Diagnostic Tools: dig and iperf

Two additional diagnostic commands extend the operator toolkit:

- **dig**: DNS query tool that resolves hostnames against a configured DNS server, reports query time in milliseconds, and displays response status codes (NOERROR, NXDOMAIN, timeout). Uses a state-machine implementation that drives simulation stepping.
- **iperf**: Throughput measurement tool that generates constant-bit-rate traffic between two endpoints. Reports per-interval statistics (bytes sent, bits/second) and total flow summary. Configurable duration, reporting interval, packets/second, and packet size.

9.8 Static Analysis: Path Tracing, Diffing, and Capacity

netsim provides several static analysis tools for "Day 2" operations, enabling operators to analyse routing state without injecting packets:

- **Path Tracing**: Computes hop-by-hop forwarding paths by walking the FIB/LFIB. Identifies routing loops, blackholes, and asymmetric routing. Traces handle complex overlays (MPLS, VXLAN) and recursive next-hops natively.
- **Topology Diffing**: Compares two routing matrices (e.g., before and after a planned configuration change) to quantify the blast radius. It highlights exactly which subnets shifted paths, providing empirical proof of change impact.
- **Capacity Planning**: Predicts network congestion by overlaying a traffic matrix onto the current routing paths. Flags links that exceed their configured bandwidth, helping to catch bottlenecks before they cause packet loss.

> **Design Decision**
>
> **Why static path tracing instead of packet-based traceroute?**
> Traceroute (above) injects real ICMP packets and advances the simulation — it shows the *actual* forwarding path including ECMP hash selection. Path tracing walks the FIB *without* advancing the simulation — it shows *all possible* paths and detects anomalies like loops and blackholes that traceroute might miss (a packet would simply be dropped). The two tools are complementary: traceroute for specific-flow validation, path tracing for exhaustive analysis.

9.9 JSON Structured Output

netsim provides structured JSON output at two levels:

Simulation-level JSON. `netsim run --format json` produces a `SimulationResult` object containing:

```
{
  "simulation": {
    "name": "dc-fabric",
    "converged": true,
    "converged_at_tick": 847,
    "final_tick": 1347
  },
  "commands": [
    {"tick": 947, "device": "PE1",
     "command": "show bgp summary", "output": "...",
     "exit_code": 0}
  ],
  "assertions": [
    {"message": "h1 -> h2 reachable", "success": true}
  ],
  "errors": []
}
```

Command-level JSON. Individual `show` commands support `--json` or a trailing `json` keyword for structured output. This enables programmatic integration—a CI script can parse `show ipv6 route --json` output directly rather than screen-scraping ASCII tables.

> **Warning**
>
> JSON output is not yet uniformly implemented across all `show` commands. `show evpn ethernet-segment json` and `show ipv6 [route|neighbors] --json` have full JSON support. Other commands currently render ASCII tables via `.to_table()`; JSON variants are being added incrementally.

9.10 Command Determinism

Commands received via gRPC are buffered and drained at phase P0 of the tick pipeline. This ensures that the simulation state observed by a command is always the state at a tick boundary, never a mid-tick intermediate state. Two users attaching simultaneously see consistent results because command delivery is serialised within the tick loop.

9.11 TUI Device Selector

Running `netsim` with no arguments launches a terminal user interface that presents a nested menu:

1. **Daemon selection**: Lists all running daemons (discovered from `.netsim/*.sock` files) with their status.
2. **Device selection**: Queries the selected daemon's `Status` RPC for its device list, grouped by type (routers, hosts, switches).
3. **REPL**: Opens an interactive session on the selected device with full tab completion.

This provides a zero-configuration discovery workflow: start a daemon, then run `netsim` and navigate to any device without remembering daemon names or device identifiers.

10 Usage

This section provides end-to-end examples of increasing complexity.

10.1 Example 1: Simple Reachability Test

Listing 6: Minimal reachability test.

```yaml
devices:
  - name: r1
    type: router
    interfaces:
      - name: eth0
        ipv4: 10.0.1.1/24
  - name: h1
    type: host
    interfaces:
      - name: eth0
        ipv4: 10.0.1.10/24
        gateway: 10.0.1.1
links:
  - endpoints: [r1:eth0, h1:eth0]
assertions:
  - type: reachability
    source: h1
    destination: 10.0.1.1
    expected: true
```

Question answered: "Can host h1 reach router r1?"

Run with: `netsim run topology.yaml`. The output reports assertion results (pass/fail) and convergence timing.

10.2 Example 2: OSPF Reconvergence After Link Failure

Listing 7: OSPF reconvergence test (abbreviated).

```
# 5-router OSPF ring: r1--r2--r3--r4--r5--r1
devices:
  - name: r1
    type: router
    # ... interfaces with OSPF area 0
  # ... r2 through r5

events:
  - at: converged + 100
    action: link_down
    link: r1-r2

assertions:
  - type: reachability
    source: h1
    destination: h2
    expected: true  # via alternate path
  - type: convergence_time
    max_ticks: 200  # must reconverge within 200 ticks
```

Question answered: "Does traffic reroute after a link failure, and does it reconverge within 200 ticks?"

10.3 Example 3: BGP L3VPN with Post-Convergence Inspection

Listing 8: BGP VPN with convergence-relative scripting.

```
# PE/CE topology with VRFs and iBGP route reflection
script:
  - at: converged + 500
    device: CE1
    command: ping 10.200.1.10
  - at: converged + 1s
    device: PE1
    command: show bgp vpnv4 vrf CUSTOMER-A
```

Question answered: "Are VPN prefixes reachable after convergence? What does the VPNv4 table look like?"

10.4 Example 4: Chaos Engineering

Listing 9: Probabilistic failure with cascade rules.

```
failure_patterns:
  - name: chaos-monkey
    trigger: !probabilistic
      check_interval: 100
      probability: 0.1
    selector:
      by_tier: spine
    action: fail_random_link
    recovery: !after_ticks
      ticks: 50

cascade_rules:
  - name: dual-failure-cascade
    trigger: !dependency_based
      primary_selector:
        link_name_pattern: "spine-*"
```

```
    action: fail_devices

assertions:
  - type: reachability
    source: host1
    destination: host2
    expected: true  # survives chaos
```

Question answered: "Does the fabric survive random spine failures with cascading effects?"

10.5 Example 5: VRRP Gateway Redundancy

Listing 10: VRRP first-hop redundancy with tracked uplinks.

```
devices:
  - name: r1
    type: router
    interfaces:
      - name: eth0
        ipv4: 10.0.1.2/24
      - name: eth1
        ipv4: 10.0.100.1/24  # uplink
  - name: r2
    type: router
    interfaces:
      - name: eth0
        ipv4: 10.0.1.3/24
      - name: eth1
        ipv4: 10.0.200.1/24  # uplink
  - name: h1
    type: host
    interfaces:
      - name: eth0
        ipv4: 10.0.1.10/24
        gateway: 10.0.1.1     # virtual IP

vrrp:
  groups:
    - id: 1
      interface: eth0
      virtual_ip: 10.0.1.1
      routers:
        - device: r1
          priority: 150       # master
          track_interface: eth1
          track_decrement: 60
        - device: r2
          priority: 100       # backup

events:
  - at: converged + 100
    action: interface_down
    device: r1
    interface: eth1           # tracked uplink fails

assertions:
  - type: reachability
    source: h1
    destination: 10.0.100.1
    expected: false           # r1 uplink is down
  - type: reachability
    source: h1
    destination: 10.0.200.1
    expected: true            # r2 takes over as master
```

Question answered: "When the primary gateway's uplink fails, does VRRP failover to the backup and

restore host reachability?"

10.6 Example 6: DHCP Relay Across Subnets

Listing 11: DHCP relay with Option 82.

```yaml
devices:
  - name: r1
    type: router
    interfaces:
      - name: eth0
        ipv4: 10.0.1.1/24
        helper_addresses: [10.0.100.10]
      - name: eth1
        ipv4: 10.0.100.1/24
  - name: dhcp-server
    type: host
    interfaces:
      - name: eth0
        ipv4: 10.0.100.10/24
        gateway: 10.0.100.1
  - name: client
    type: host
    interfaces:
      - name: eth0
        ipv4: 10.0.1.0/24    # unconfigured
        gateway: 10.0.1.1
```

Question answered: "Does the relay agent correctly forward DHCP Discover packets from the client subnet to the server, populating GIADDR and Option 82?"

11 Example Topologies

netsim ships with over 40 example topologies, embedded in the CLI binary via `include_str!()`. They serve three purposes: onboarding (learning the YAML format), regression testing (every example is gated by a convergence test), and benchmarking (the larger examples are used for performance measurement). This section surveys the examples by category.

11.1 Built-in Examples (`netsim example`)

The `netsim example <name>` subcommand prints a complete, runnable topology to stdout. Eight examples are available:

Table 11: Built-in example topologies.

Name	Protocols	Devices	Demonstrates
`simple`	—	2	Minimal connectivity
`ospf-triangle`	OSPF	5	Basic IGP routing
`data-center`	OSPF	20	Spine-leaf L3 fabric
`wan-mesh`	OSPF	18	Full-mesh WAN with costs
`enterprise-campus`	OSPF	18	3-tier campus hierarchy
`mixed-network`	OSPF	15	Router/Switch/Hub mix
`large-enterprise`	OSPF, Traffic	440	Multi-area enterprise
`service-provider`	OSPF, Traffic	108	P/PE/CE hierarchy

Example: Generate and run a topology

```
# Print the service-provider example
netsim example service-provider > sp.yaml
```

```
# Run it
netsim run sp.yaml --format json --output sp-results.json
```

11.2 Extended Examples

The `examples/` directory contains 40+ topologies covering specialised protocols and advanced scenarios. Table 12 highlights the most interesting ones, grouped by the protocol stack they exercise.

Table 12: Selected extended example topologies.

Example	Protocols	Dev.	Demonstrates
IGP & Link-State Routing			
`isis-hierarchical`	IS-IS	8	L1/L2/L1L2 areas
`isis-lan`	IS-IS	4	Broadcast LAN DIS election
`scale-test-ring`	OSPF	100	100-device ring convergence
BGP & Policy			
`bgp-ipv6-multi-as`	BGP	3	Multi-AS IPv6 peering
`bgp-community-policy`	OSPF, BGP	3	NO_EXPORT, NO_ADVERTISE
`docs-bgp-rr-loop`	BGP	3	iBGP route reflection loops
MPLS, SR & Tunneling			
`docs-mpls-ldp-oam`	OSPF, MPLS/LDP	5	LSP ping/traceroute
`rsvp-te-tunnel`	OSPF, RSVP-TE	4	Explicit path tunnels
`sr-mpls-transport`	OSPF, SR-MPLS	6	Segment Routing with SRGB
`gre-overlay`	OSPF, GRE	5	GRE over OSPF underlay
L3VPN & Data Centre			
`l3vpn-service-provider`	OSPF, BGP, MPLS	7	VRF, VPNv4, PE/CE
`dc-fabric-irb`	OSPF, BGP, VXLAN	8	Integrated routing/bridging
`dc-fabric-multi-homing`	BGP	10	EVPN multi-homing
`dual-stack-data-center`	OSPF	20	IPv4 + IPv6 fabric
Enterprise & Campus			
`vrrp-redundancy`	VRRP	3	First-hop gateway failover
`dhcp-relay-simple`	DHCP Relay	3	Broadcast-to-unicast relay
`rip-simple-network`	RIP	4	Distance-vector convergence
Fast Failover & Infrastructure			
`bfd-fast-failover`	OSPF, BGP, BFD	3	Sub-second BFD failover
`lag-lacp`	LACP	4	Dynamic LAG negotiation
Traffic & Chaos			
`dc-fabric-traffic`	OSPF, Traffic	10	Traffic generation in DC
`qos-congestion`	OSPF, Traffic	4	Congestion & queue drops
`chaos-simple`	—	6	Failure injection patterns
`datacenter-disaster`	—	10	Multi-failure disaster
`jitter-validation`	—	6	Impairment validation
Dual-Stack & Scale			
`dual-stack-service-provider`	OSPF, Traffic	108	IPv4+IPv6 SP

Example	Protocols	Dev.	Demonstrates
`large-enterprise`	OSPF, Traffic	440	Multi-area enterprise

Note

Every example in this table is covered by an automated regression test in `tests/example_topology_regression.rs`. Each test verifies that the topology parses, builds, and converges within 50,000 ticks with a convergence threshold of 500 stable ticks. This ensures that protocol changes never silently break existing topologies.

11.3 Protocol Coverage Across Examples

Table 13 shows which protocols are exercised by the example suite. The combination ensures that every protocol implementation has at least one end-to-end topology test.

Table 13: Protocol coverage across the example suite.

Protocol	Examples	Key topologies
OSPF v2	20+	`ospf-triangle`, `large-enterprise`
OSPFv3	2	`dual-stack-*`
IS-IS	3	`isis-hierarchical`, `isis-lan`
BGP (iBGP)	6	`docs-bgp-rr-loop`, `l3vpn-*`
BGP (eBGP)	3	`bgp-ipv6-multi-as`, `dc-fabric`
BGP EVPN	3	`dc-fabric-irb`, `dc-fabric-multi-homing`
MPLS/LDP	3	`docs-mpls-ldp-oam`, `l3vpn-*`
RSVP-TE	1	`rsvp-te-tunnel`
SR-MPLS	1	`sr-mpls-transport`
BFD	1	`bfd-fast-failover`
LACP	2	`lag-lacp`, `lag-basic`
GRE	1	`gre-overlay`
VXLAN	2	`dc-fabric-irb`, `dc-fabric-multi-homing`
RIP	1	`rip-simple-network`
STP	1	`stp-loop-prevention`
VRRP	1	`vrrp-redundancy`
DHCP Relay	1	`dhcp-relay-simple`
Traffic gen	8	`*-traffic`, `large-enterprise`

12 Performance

This section documents netsim's performance characteristics, benchmarking infrastructure, and scalability.

12.1 Benchmark Infrastructure

netsim includes a Criterion-based benchmark suite in `benches/` that measures simulation performance across topology sizes and workload types. Three benchmark suites target different aspects:

Table 14: Benchmark suites.

Suite	File	What it measures
Scale	`benches/scale.rs`	Throughput (ticks/sec) across topology sizes from 2 to 486 devices. Tests five workload types: ControlPlane, Mixed, DataPlane, DataPlaneMultiFlow, MixedWithFailure.
Realistic	`benches/realistic.rs`	Parallel vs. serial execution comparison. Tracks ticks/sec, packet loss %, throughput (pps), and latency percentiles (P50/P95/P99).
Dual-stack	`benches/dual_stack_scale.rs`	IPv4 vs. IPv6 convergence overhead. Expected: dual-stack adds $< 1.5\times$ overhead vs. single-stack.

Scenario registry. Benchmarks draw from a scenario registry that defines four reference topologies at different scales:

Table 15: Benchmark reference scenarios.

Scenario	Devices	Topology	Purpose
`two-host-wire`	2	Direct link	Baseline overhead
`three-host-chain`	3	Linear chain	Minimal forwarding path
`wan-mesh-150`	150	25 routers, 5 hosts each	Medium-scale WAN
`dc-fabric-486`	486	6 spines, 30 leaves, 15 hosts/leaf	Large-scale DC fabric

12.2 Metrics

Each benchmark run collects six metrics:

1. **Ticks/sec**: Simulation throughput—how many ticks the engine can execute per wall-clock second. Higher is better.
2. **Convergence tick**: The tick at which the convergence predicate (section 4) first becomes true. Lower means faster protocol convergence.
3. **Packet loss %**: Fraction of generated traffic that is dropped. Used to validate data-plane correctness under failure scenarios.
4. **Throughput (pps)**: Packets delivered per second of simulation time. Measures data-plane capacity.
5. **Latency percentiles**: P50, P95, P99 packet delivery times. Measures forwarding-path quality.
6. **Determinism variance**: Standard deviation of convergence tick across repeated runs. Should be zero for a deterministic engine.

> **Note**
>
> Benchmarks default to 12 repetitions (`DEFAULT_REPEATS: 12`) and a 500-tick budget (`DEFAULT_TICK_BUDGET: 500`). The 12-repetition count is chosen to detect non-determinism: if any run produces a different convergence tick, the benchmark flags a determinism warning.

12.3 Scalability

netsim's performance scales with topology size across four tiers:

Table 16: Scalability tiers.

Tier	Devices	Example	Characteristics
Minimal	2–5	`simple`, `ospf-triangle`	Sub-millisecond per tick; useful for unit tests
Small	6–20	`data-center`, `enterprise-campus`	Milliseconds per tick; protocol interaction testing
Medium	100–150	`scale-test-ring`, `wan-mesh-150`	Tens of milliseconds per tick; regional network scale
Large	440–486	`large-enterprise`, `dc-fabric-486`	<0.5 ms per tick; adaptive parallelism engages

12.4 Adaptive Parallelism

The engine uses two thresholds to decide when parallelism is worthwhile:

- **Device threshold** (default: 500): Below this, device ticking (phase P2) runs serially. At or above, Rayon `par_iter_mut` is used.
- **Packet threshold** (default: 1,000): Below this, packet delivery (phases P1, P3, P4) runs serially. At or above, packet processing is parallelised.

> **Design Decision**
>
> **Why two thresholds?**
> Device ticking and packet processing have different computational profiles. Device ticking involves running protocol state machines (OSPF SPF, BGP decision process)—CPU-intensive with variable cost per device. Packet processing involves queue operations and frame copying—memory-intensive with uniform cost per packet. Separate thresholds allow each to switch independently based on workload.
> **Trade-off**: Both thresholds are compile-time defaults, not auto-tuned at runtime. A topology just below the threshold misses parallelism even on machines with many cores.

12.5 Execution Modes

netsim supports two execution modes, selectable at build time or via configuration:

- **Parallel** (default): Uses Rayon with auto-detected worker threads. The number of threads can be overridden via `worker_threads` in `EngineConfig`.
- **Serial**: Disables all parallelism. Useful for debugging (deterministic single-threaded execution makes backtraces readable) and for measuring parallelism speedup in benchmarks.

The Realistic benchmark suite explicitly compares parallel and serial execution on the same topology to measure Rayon overhead and parallelism benefit. This comparison is tracked across releases to detect performance regressions.

12.6 Regression Testing

All 20+ example topologies are covered by convergence regression tests.[1] Each test verifies:

1. The YAML topology parses without errors.

[1] `tests/example_topology_regression.rs`

2. The Builder constructs the simulation without errors.
3. The simulation converges within 50,000 ticks.
4. Convergence is sustained for 500 consecutive stable ticks.

These tests run in CI on every commit, ensuring that protocol changes or engine modifications never silently break convergence behaviour for any reference topology.

12.7 Control Plane Policing (CoPP)

netsim implements token-bucket rate limiting for control-plane packets, protecting routers from DoS attacks in simulation scenarios:

- **Per-protocol rate limiting**: Separate token buckets for ICMP, BGP, OSPF, and other control-plane protocols.
- **Configurable parameters**: Burst size and refill rate are tunable per protocol.
- **Statistics**: Accepted/dropped packet counters enable validation of CoPP policy effectiveness.

12.8 End-to-End Hardening

The test suite includes 20+ integration tests covering four hardening categories:

1. **Resiliency & Failure Scenarios**: RSVP-TE LSP convergence under node crash, MPLS label stack determinism under cascading failures, link flap and brown-out scenarios.
2. **Robustness & Attack Defence**: Graceful handling of malformed packets (TTL=0, cyclic labels, excessive MPLS stack depth), control-plane fuzzing, route spoofing defence.
3. **Performance & Telemetry**: High-throughput data-plane validation ensuring control-plane protocols are not starved, bounded memory growth under routing loops (TTL expiration).
4. **Control Plane Policing**: Token-bucket rate limiting validation, per-protocol burst/refill configuration, DoS resilience.

13 Limitations

13.1 No TCP/UDP Transport Modelling

Description: netsim abstracts the transport layer. There is no TCP congestion window, RTT estimation, flow control, or UDP socket modelling. Traffic generators produce frames at a configured rate without transport-layer feedback.

Workaround: For transport-layer research, use ns-3 or a container-based emulator. netsim is designed for routing-level validation, not transport-level simulation.

Planned resolution: Basic TCP throughput estimation is under consideration for a future release, but full cwnd/ssthresh modelling is out of scope.

13.2 Idealised Device Behaviour

Description: netsim models RFC behaviour, not vendor-specific implementation quirks. Real routers have bugs, non-standard defaults, and proprietary extensions that netsim does not reproduce.

Workaround: Use netsim for "does the design work in theory?" and Containerlab for "does it work with this specific NOS version?"

Planned resolution: No current plan. Modelling vendor bugs would undermine the determinism guarantee.

13.3 No Detailed Queuing Theory

Description: The `BatchQueue` is FIFO with drop-tail. There is no RED, WRED, WFQ, or priority queuing. Interface capacity tracking (Phase 118) is recent and tracks utilisation but does not model queuing delays.

Workaround: For QoS validation, use a container-based emulator with a real queuing discipline.

Planned resolution: Priority queuing and basic RED are candidates for a future milestone.

13.4 Scale Ceiling

Description: netsim has been tested to 486 devices. It is not validated at 10,000+ device scale. Memory is bounded by single-machine RAM; there is no distributed execution mode.

Workaround: For topologies larger than ~500 devices, validate representative subsets rather than the full topology.

Planned resolution: Distributed execution via partitioned tick processing is listed as future work.

13.5 EVPN Synchronisation Fidelity

Description: The snapshot-then-apply EVPN synchronisation pattern does not model individual BGP UPDATE message ordering or per-session backpressure. It cannot reproduce bugs that depend on the order in which a route reflector processes updates from different clients.

Workaround: For BGP implementation testing (as opposed to configuration validation), use real BGP daemons in containers.

Planned resolution: No current plan. The snapshot-then-apply design is a deliberate trade-off favouring determinism over message-level fidelity.

13.6 BFD Auto-Session Wiring

Description: BFD sessions configured via `bfd: true` in YAML are not automatically created for all protocol neighbours. Tests verify the BFD feature works, but full auto-wiring from the YAML flag requires additional Builder logic.

Workaround: Configure BFD sessions explicitly in the YAML topology.

Planned resolution: Deferred to a future milestone.

14 Future Work

- **Distributed execution**: Partition the tick pipeline across multiple machines for 10,000+ device topologies.
- **TCP transport modelling**: Add basic TCP throughput estimation (bandwidth-delay product) without full congestion control simulation.
- **Network digital twin integration**: Connect netsim to production network controllers to serve as a pre-deployment validation backend.
- **SRv6 and Segment Routing**: Extend the MPLS infrastructure with SRv6 (Proposed milestone v2.2).
- **Multicast**: IGMP/MLD snooping and PIM-SM (Proposed milestone v2.3).
- **Topogen integration**: Native export format for the Topogen topology generator to enable automated topology generation and simulation.
- **RAPID STP / MSTP**: Extend STP to Rapid STP (802.1w) and Multiple STP (802.1s) for faster L2 convergence.

- **Full DHCP server**: Add pool-based address allocation to complement the existing relay implementation.

References

[1] L. Andersson, I. Minei, and B. Thomas. LDP Specification. RFC 5036, 2007.

[2] Ali Basiri, Niosha Behnam, Ruud de Rooij, Lorin Hochstein, Luke Kosewski, Justin Reynolds, and Casey Rosenthal. Chaos engineering. *IEEE Software*, 33(3):35–41, 2016.

[3] R. Coltun, D. Ferguson, J. Moy, and A. Lindem. OSPF for IPv6. RFC 5340, 2008.

[4] Containerlab. Containerlab: Container-based networking labs, 2024. Accessed: 2024-01-15.

[5] R. Droms. Dynamic Host Configuration Protocol. RFC 2131, 1997.

[6] R. Hinden. Virtual Router Redundancy Protocol (VRRP). RFC 3768, 2004.

[7] D. Katz and D. Ward. Bidirectional Forwarding Detection (BFD). RFC 5880, 2010.

[8] Craig Labovitz, Abha Ahuja, Abhijit Bose, and Farnam Jahanian. Delayed internet routing convergence. In *Proc. SIGCOMM*, pages 175–187. ACM, 2000.

[9] M. Mahalingam, D. Dutt, K. Duda, P. Agarwal, L. Kreeger, T. Sridhar, M. Bursell, and C. Wright. Virtual eXtensible Local Area Network (VXLAN). RFC 7348, 2014.

[10] G. Malkin. RIP Version 2. RFC 2453, 1998.

[11] G. Malkin and R. Minnear. RIPng for IPv6. RFC 2080, 1997.

[12] Nicholas D. Matsakis and Felix S. Klock. The Rust language. In *Proc. HILT*, pages 103–104. ACM, 2014.

[13] J. Moy. OSPF Version 2. RFC 2328, 1998.

[14] S. Nadas. Virtual Router Redundancy Protocol (VRRP) Version 3 for IPv4 and IPv6. RFC 5798, 2010.

[15] Y. Rekhter, T. Li, and S. Hares. A Border Gateway Protocol 4 (BGP-4). RFC 4271, 2006.

[16] A. Sajassi, R. Aggarwal, N. Bitar, A. Isaac, J. Uttaro, J. Drake, and W. Henderickx. BGP MPLS-Based Ethernet VPN. RFC 7432, 2015.

A YAML Schema Quick Reference

Top-level key	Type	Description
`name`	String	Topology name
`tick_ms`	Integer	Tick duration in ms (default: 1)
`devices`	List	Device definitions
`links`	List	Link definitions
`ospf`	Object	OSPF configuration
`bgp`	Object	BGP configuration
`isis`	Object	IS-IS configuration
`mpls`	Object	MPLS/LDP configuration
`rsvp`	Object	RSVP-TE configuration
`rip`	Object	RIP/RIPng configuration
`stp`	Object	STP configuration
`vrrp`	Object	VRRP group configuration
`traffic`	List	Traffic generator definitions
`impairment_profiles`	List	Named impairment profiles
`failure_patterns`	List	Chaos engineering patterns
`events`	List	Scheduled events
`assertions`	List	Post-simulation assertions
`script`	List	Convergence-relative commands

B CLI Command Quick Reference

Command	Description
`show ip route`	Display IPv4 FIB
`show ip route vrf NAME`	Display VRF-specific FIB
`show bgp summary`	Display BGP neighbour summary
`show bgp vpnv4 vrf NAME`	Display VPNv4 routes for a VRF
`show bgp evpn`	Display EVPN routes
`show ospf neighbor`	Display OSPF adjacencies
`show ospf database`	Display OSPF LSDB
`show isis adjacency`	Display IS-IS adjacencies
`show mpls forwarding`	Display MPLS LFIB
`show mpls ldp bindings`	Display LDP label bindings
`show interface`	Display interface status
`show lldp neighbors`	Display LLDP neighbour table
`show lacp`	Display LACP state
`show bfd sessions`	Display BFD session state
`show rip`	Display RIP routing table and neighbours
`show stp`	Display STP bridge and port state
`show vrrp`	Display VRRP group state and priority
`show vxlan vtep`	Display VXLAN VTEP endpoints
`show ip dhcp relay`	Display DHCP relay statistics
`show ipv6 route`	Display IPv6 FIB
`show ospfv3 neighbor`	Display OSPFv3 adjacencies
`show bgp ipv6 unicast`	Display BGP IPv6 routes
`show routing-matrix`	Display full routing matrix
`show trace DESTINATION`	Display static forwarding path

Command	Description
`ping DESTINATION`	ICMP ping (drives simulation stepping)
`traceroute DESTINATION`	Traceroute (drives simulation stepping)
`dig HOSTNAME`	DNS query (drives simulation stepping)
`iperf TARGET`	Throughput measurement

All `show` commands support `--json` for structured output.